

Labwork 5

Le Quoc Trung, 2540103

October 2, 2026

0.1 Goal

Turn the grayscale kernel of labwork 4 into a 7x7 Gaussian blur convolution, write it twice, once reading the filter from global memory and once copying the filter into shared memory first, then compare the two for different block sizes.

Run on Google Colab with a Tesla T4, same image as before, 491x553, so 271523 pixels. The filter is the 7x7 table from the lecture, whose coefficients add up to 1003.

0.2 Implementation

Same structure as labwork 4: a 2D grid, one thread per pixel, x for the column and y for the row. The difference is that a thread now walks the 7x7 window centred on its pixel, multiplies each neighbour by the matching coefficient, sums everything and divides by the total weight, for the three channels separately.

Near the borders part of the window falls outside the image, so I only add up the coefficients that were actually used instead of dividing by 1003. Otherwise the first three rows and columns come out darker.

The second version copies the filter into shared memory first, then calls `cuda.syncthreads()` so the whole block waits for the copy. The slide writes `cuda.synchronize()`, but that one is for the host; inside a kernel the barrier is `cuda.syncthreads()`.

Both kernels give the same image as the CPU version.

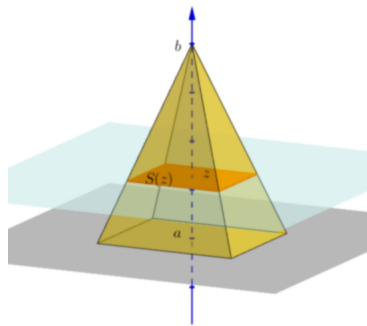


Figure 1: Output of the blur kernel

0.3 Results

The CPU version takes 42.04 s. Each GPU value is the average of 10 launches after one warm-up launch.

Block size	Threads	Global (ms)	Shared (ms)	Best speedup
4x4	16	1.083	1.154	38823
8x8	64	0.600	0.606	70111
16x16	256	0.490	0.535	85864
32x32	1024	0.520	0.525	80856
32x4	128	0.485	0.530	86628
32x8	256	0.498	0.537	84357
32x16	512	0.507	0.533	82932
16x8	128	0.479	0.517	87793
8x16	128	0.601	0.609	69966

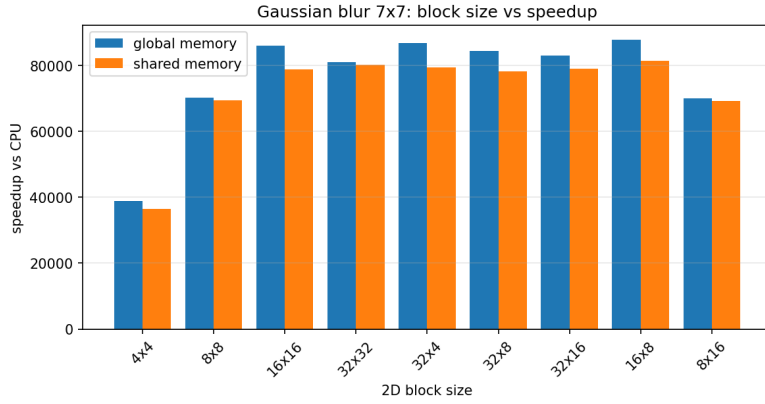


Figure 2: Block size vs speedup, with and without shared memory

The best result is 16x8 without shared memory, 87793 times faster than the CPU. Shared memory is slightly slower than global memory in every configuration.